NEXT ENGINEER · _SHEERAT_

Programming Languages (C and Python)

ACS101

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 1 | **Credits:** 3

L-T-P: 3-0-0 | **Scheme:** AKTU B.Tech 1st year, CSE stream, 2026-27 onwards

UNIT 1 C Programming Basics

UNIT 2 Control Structures and Functions in C

UNIT 3 Arrays, Strings, Pointers, Structures and Files in C

UNIT 4 Introduction to Python

UNIT 5 Python Data Structures, Files and Libraries

Original revision notes written for the AKTU syllabus. Units 1 to 3 cover C and Units 4 and 5 cover Python. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: syntax, rules and comparison tables.
3. **Programs and output questions**: short programs that are asked again and again, with their output checked by running them.
4. **Key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (structure of a C program, data types, operators, precedence, printf and scanf) and Unit 2 (if, switch, loops, functions, recursion). Write each standard program once by hand.
- **Afternoon (3 hours):** Unit 3 (arrays, strings, pointers, structures, files). Then Unit 4 (Python basics, control flow, functions).
- **Evening (2 hours):** Unit 5 (list, tuple, dict, set, files, CSV, NumPy, Matplotlib), then the "Last-hour checklist".
- **In the exam:** write programs with correct headers, indentation and comments, and show a sample input and output. For output questions, trace line by line and write the values of variables in the margin.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Expect program-writing questions in both C and Python. Internal assessment carries another 30 marks.

UNIT 1 C Programming Basics

✍️ 2-mark definitions

- **Algorithm:** a finite, ordered set of clear steps to solve a problem. **Flowchart:** a diagram of an algorithm using standard symbols (oval for start and end, parallelogram for input and output, rectangle for processing, diamond for decision).
- **Compiler:** translates the whole source program into machine code before running (C). **Interpreter:** translates and runs one statement at a time (Python).
- **Token:** the smallest unit of a C program: keywords, identifiers, constants, strings, operators and special symbols.
- **Identifier:** a name given to a variable or function. It starts with a letter or underscore, has no spaces, and is case sensitive.
- **Keyword:** a reserved word with fixed meaning (int, float, if, while, return, and so on; C has 32 in C89).
- **Variable:** a named memory location whose value can change. **Constant:** a value that does not change (`const int N = 10;` or `#define PI 3.14`).
- **Type conversion:** implicit (automatic, smaller type promoted to larger, as in int + float gives float) or explicit (type casting, as in `(float)a/b`).
- **Header file:** a file of declarations included with `#include` , such as stdio.h for printf and scanf, math.h for sqrt and pow.

💡 Core concepts

Structure of a C program:

1. Documentation section (comments).
2. Preprocessor section (`#include` , `#define`).
3. Global declarations.
4. `main()` function: declarations, then statements.
5. User-defined functions.

```c
#include <stdio.h>      /* preprocessor directive */
int main(void) {        /* execution starts here */
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);   /* & gives the address */
    printf("Sum = %d\n", a + b);
    return 0;           /* 0 means success */
}
```

Compile and execute steps: edit (prog.c) → preprocess (expand includes and macros) → compile (to assembly and object code, prog.o) → link (add library code) → load and run (a.out or prog.exe). With gcc: `gcc prog.c -o prog` then `./prog` .

Basic data types (typical 32-bit or 64-bit system):

Type	Size	Format specifier	Example
char	1 byte	%c	'A'
int	4 bytes	%d	25
float	4 bytes	%f	3.14
double	8 bytes	%lf	3.14159265
long int	4 or 8 bytes	%ld	100000L
unsigned int	4 bytes	%u	40000

Operators:

Category	Operators	Note
Arithmetic	+ − * / %	Integer / drops the fraction; % only for integers
Relational	< <= > >= == !=	Result is 1 (true) or 0 (false)
Logical	&& || !	Short-circuit evaluation
Assignment	= += −= *= /= %=	a += 2 means a = a + 2
Increment, decrement	++ −−	Prefix changes first, postfix uses first
Bitwise	& | ^ ~ << >>	Work on bits; x << 1 doubles x
Conditional	?:	max = (a > b) ? a : b
Others	sizeof, comma, & (address), * (value at)	

- **Precedence (high to low, short form):** `() [] ->` → unary (`! ++ -- sizeof`) → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment → comma.
- **Associativity:** most operators are left to right; unary, conditional and assignment are right to left.

Programs and output questions

Output question 1.

```
int a = 7, b = 2;
printf("%d %d %.2f %d", a / b, a % b, (float)a / b, 5 + 3 * 2 > 10 && 4 != 4);
```

Output: **3 1 3.50 0**. (5 + 6 = 11 > 10 is true, but 4 != 4 is false, so && gives 0.)

Output question 2.

```
int i = 10;
char c = 'A' + 2;
printf("%d %d %d %c", i << 1, i >> 1, 'A' + 2, c);
```

Output: **20 5 67 C** (ASCII of 'A' is 65).

Output question 3. `printf("%d", 17 / 3 * 3 + 17 % 3);` gives 5 × 3 + 2 = **17**.

Program (swap two numbers without a third variable):

```
a = a + b;  b = a - b;  a = a - b;
```

Key points to remember

- Compiler vs interpreter; edit → preprocess → compile → link → run.
- Sizes: char 1, int 4, float 4, double 8; format specifiers %c %d %f %lf.
- Integer division truncates; % needs integers; relational results are 1 or 0.
- Postfix uses the old value; prefix changes first. Do not use x++ and ++x in the same expression (undefined behaviour).
- `scanf` needs `&` before ordinary variables.

⭐ Important questions

1. Explain the structure of a C program with an example. Explain the steps to compile and execute it.
2. Explain the basic data types of C with size, range and format specifier.
3. Explain the operators of C with examples. What are precedence and associativity?
4. What is type conversion? Explain implicit and explicit conversion with examples.
5. Draw a flowchart and write a program to find the roots of a quadratic equation (or the largest of three numbers).

UNIT 2 Control Structures and Functions in C

✍️ 2-mark definitions

- **Control statement:** a statement that changes the order of execution: decision (if, if-else, else-if ladder, switch), loop (for, while, do-while) and jump (break, continue, goto, return).
- **Entry-controlled loop:** the condition is checked before the body (for, while), so the body may run zero times. **Exit-controlled loop:** the condition is checked after the body (do-while), so the body runs at least once.
- **break:** leaves the nearest loop or switch at once. **continue:** skips the rest of the current iteration and goes to the next one.
- **Function:** a named, self-contained block that does one task. It has a declaration (prototype), a definition and a call.
- **Call by value:** a copy of the argument is passed; changes inside the function do not affect the caller (the default in C).
- **Call by reference:** the address is passed using pointers, so the function can change the caller's variable.
- **Recursion:** a function calling itself, with a **base case** that stops it.
- **Storage classes:** auto (local, default), register (hint to keep in a register), static (keeps its value between calls), extern (global defined elsewhere).

💡 Core concepts

switch rules: the expression must be int or char; case labels are constants; without `break`, execution falls through to the next case; `default` handles all other values.

Loop comparison:

Feature	for	while	do-while
Type	Entry controlled	Entry controlled	Exit controlled
Minimum runs	0	0	1
Best when	Count is known	Condition based	Menu programs, input checking
Semicolon after condition	No	No	Yes: `while(cond);`

Function parts:

```c
int add(int, int);              /* prototype (declaration) */
int main(void) {
    printf("%d", add(3, 4));    /* call: actual arguments */
    return 0;
}
int add(int x, int y) {         /* definition: formal parameters */
    return x + y;
}
```

Call by value vs call by reference (swap):

```c
void swapV(int a, int b)   { int t = a; a = b; b = t; }      /* caller unchanged */
void swapR(int *a, int *b) { int t = *a; *a = *b; *b = t; }  /* caller swapped  */
/* call: swapR(&x, &y); */
```

Recursion vs iteration: recursion is shorter and natural for factorial, Fibonacci and trees, but uses stack memory for every call and is slower. Iteration uses a loop and constant memory.

Programs

Factorial and Fibonacci (recursive):

```c
int fact(int n) { return (n <= 1) ? 1 : n * fact(n - 1); }   /* fact(5) = 120 */
int fib(int n)  { return (n < 2) ? n : fib(n - 1) + fib(n - 2); } /* fib(10) = 55 */
```

Armstrong number (153 = $1^3 + 5^3 + 3^3$):

```c
int n = 153, s = 0, t = n;
while (t != 0) { int d = t % 10; s += d * d * d; t /= 10; }
printf(s == n ? "Armstrong" : "Not Armstrong");   /* prints Armstrong */
```

Prime check:

```c
int n, i, prime = 1;
scanf("%d", &n);
if (n < 2) prime = 0;
for (i = 2; i * i <= n; i++)
    if (n % i == 0) { prime = 0; break; }
printf(prime ? "Prime" : "Not prime");
```

Calculator using switch:

```c
char op; float a, b;
scanf("%f %c %f", &a, &op, &b);
switch (op) {
    case '+': printf("%.2f", a + b); break;
    case '-': printf("%.2f", a - b); break;
    case '*': printf("%.2f", a * b); break;
    case '/': if (b != 0) printf("%.2f", a / b); else printf("Divide by zero"); break;
    default:  printf("Invalid operator");
}
```

Other programs to practise: reverse a number, sum of digits, palindrome number, GCD by Euclid's method, pattern printing with nested loops, even or odd, leap year (divisible by 4 and not by 100, or divisible by 400).

Key points to remember

- switch works on int or char; remember `break`.
- do-while runs at least once and ends with a semicolon.
- C passes by value; use pointers for call by reference.
- Every recursion needs a base case; fact(5) = 120, fib(10) = 55.
- static local variables keep their value between calls.

⭐ Important questions

1. Explain decision-making statements in C (if, else-if ladder, switch) with programs.
2. Compare for, while and do-while loops with syntax and examples. Explain break and continue.
3. Explain call by value and call by reference with a swap program.
4. What is recursion? Write recursive programs for factorial and Fibonacci; compare recursion with iteration.
5. Write programs: prime number, Armstrong number, palindrome, a menu-driven calculator.

NEXT ENGINEER _sheerat_

UNIT 3 Arrays, Strings, Pointers, Structures and Files in C

✍️ 2-mark definitions

- **Array:** a collection of elements of the same type in contiguous memory, accessed by index from 0. `int a[5];`
- **2D array:** an array of arrays, stored row by row (row-major). `int m[3][3];`
- **String:** a character array ending with the null character `'\0'`. `char s[] = "AKTU";` uses 5 bytes.
- **Pointer:** a variable that stores the address of another variable. `int *p = &x;` and `*p` gives the value of x.
- **Pointer arithmetic:** p + 1 moves by sizeof(type) bytes; `a[i]` is the same as `*(a + i)`.
- **Structure:** a user-defined type grouping variables of different types under one name. **Union:** like a structure, but all members share the same memory, so its size is that of the largest member.
- **File:** data stored permanently on disk. A `FILE *` pointer is used with fopen, fclose, fprintf, fscanf, fgetc and fputc.
- **Dynamic memory allocation:** getting memory at run time with malloc, calloc, realloc, and returning it with free.

💡 Core concepts

Common string functions (string.h):

Function	Work
strlen(s)	Length without '\0'
strcpy(d, s)	Copy s to d
strcat(d, s)	Join s to the end of d
strcmp(a, b)	0 if equal, negative or positive otherwise
strrev(s)	Reverse (not standard; available in some compilers)

Structure vs union:

Feature	Structure	Union
Memory	Sum of members (plus padding)	Size of largest member
Members active	All at once	One at a time
Keyword	struct	union

File modes: "r" read, "w" write (creates or empties), "a" append, "r+" read and write, "rb" or "wb" for binary files.

malloc vs calloc: malloc(n × size) leaves garbage values; calloc(n, size) sets all bytes to zero. Both return NULL if memory is not available.

Programs

Largest element of an array:

```
int a[5] = {3, 8, 1, 9, 4}, max = a[0];
for (int k = 1; k < 5; k++) if (a[k] > max) max = a[k];
printf("%d", max);   /* 9 */
```

Matrix addition and transpose:

```
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        sum[i][j] = A[i][j] + B[i][j];
        T[j][i]   = A[i][j];     /* transpose */
    }
```

Count vowels in a string (without library functions):

```
char s[100]; int i, v = 0;
fgets(s, sizeof s, stdin);
for (i = 0; s[i] != '\0'; i++)
    if (strchr("aeiouAEIOU", s[i])) v++;
printf("Vowels = %d", v);
```

Pointer output question.

```
int p = 10, *q = &p;
*q = 20;
printf("%d", p);     /* 20: p was changed through the pointer */
```

Structure for students:

```
struct student { int roll; char name[30]; float marks; };
struct student s[3];
for (i = 0; i < 3; i++) scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
```

Write and read a file:

```
FILE *fp = fopen("data.txt", "w");
fprintf(fp, "Next Engineer %d\n", 2026);
fclose(fp);
fp = fopen("data.txt", "r");
char line[50];
while (fgets(line, sizeof line, fp)) printf("%s", line);
fclose(fp);
```

Key points to remember

- Index starts at 0; a[i] = *(a + i); strings end with '\0'.
- `&` gives the address; `*` gives the value at the address.

- Union size = largest member; structure size = sum plus padding.
- Always fclose a file and free malloc'd memory; check for NULL.

⭐ Important questions

1. What is an array? Explain 1D and 2D arrays; write programs for matrix addition, multiplication or transpose.
2. Explain strings and string functions; write a program to check a palindrome string without library functions.
3. What is a pointer? Explain pointer arithmetic and the relation between arrays and pointers.
4. Explain structure and union with examples and differences. Write a program to store and print student records.
5. Explain file handling in C with modes and functions; write a program to copy one file into another.

NEXT ENGINEER _sheerat_

UNIT 4 Introduction to Python

✍️ 2-mark definitions

- **Python:** a high-level, interpreted, dynamically typed, general-purpose language with simple syntax, created by Guido van Rossum (1991).
- **Why Python:** easy to read, large standard library, huge ecosystem (NumPy, Pandas, Matplotlib, TensorFlow), works on every platform, used in AI, data science, web and automation.
- **Dynamic typing:** the type is attached to the value, not the variable; `x = 5` then `x = "five"` is allowed.
- **Indentation:** Python uses indentation (usually 4 spaces), not braces, to mark blocks. Wrong indentation is an error.
- **Mutable vs immutable:** lists, dicts and sets can be changed in place (mutable); int, float, str and tuple cannot (immutable).
- **Function:** defined with `def`; can return several values (as a tuple); can have default, keyword and variable-length arguments.
- **Default argument:** a parameter with a default value used when the caller does not pass it.

💡 Core concepts

Data types: int, float, complex, bool (True, False), str, list, tuple, dict, set, NoneType. Use `type(x)` to check.

Input and output:

```
name = input("Name: ")            # always returns a string
age = int(input("Age: "))         # convert explicitly
print("Hello", name, "you are", age)
print(f"Next year you will be {age + 1}")   # f-string
```

Operators that differ from C:

Operator	Meaning	Example
/	True division	7 / 2 = 3.5
//	Floor division	7 // 2 = 3, −7 // 2 = −4
**	Power	2 ** 10 = 1024
and, or, not	Logical	(a > 0) and (b > 0)
in, not in	Membership	'a' in 'cat' is True
is, is not	Identity	x is None

Control flow:

```
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
else:
    grade = "C"

for i in range(1, 6):     # 1 to 5
    print(i, end=" ")

n = 5
while n > 0:
    n -= 1
```

- `range(start, stop, step)` excludes stop. Loops can have an `else` part that runs when the loop ends without `break`.

C vs Python:

Feature	C	Python
Translation	Compiled	Interpreted
Typing	Static	Dynamic
Blocks	Braces { }	Indentation
Memory	Manual (malloc, free)	Automatic (garbage collection)
Speed	Fast	Slower
Pointers	Yes	No
Use	Systems, embedded	AI, data, scripting, web

Programs and output questions

Functions with default arguments:

```
def greet(name, msg="Hello"):
    return f"{msg}, {name}!"
print(greet("Asha"), greet("Ravi", "Hi"))
```

Output: **Hello, Asha! Hi, Ravi!**

Output question. `print(7 // 2, 7 % 2, 7 / 2, 2 ** 10, -7 // 2)` gives **3 1 3.5 1024 −4**.

Factorial, prime and Fibonacci in Python:

```python
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

a, b = 0, 1
for _ in range(10):
    print(a, end=" ")      # 0 1 1 2 3 5 8 13 21 34
    a, b = b, a + b
```

Key points to remember

- input() returns str; convert with int() or float().
- / gives float, // floors, ** is power.
- Indentation defines blocks; elif, not else if.
- range(stop) starts at 0 and excludes stop.
- Functions can return several values: `return a, b`.

⭐ Important questions

1. What are the features of Python? Compare Python with C.
2. Explain Python data types and operators with examples (including //, ** and membership operators).
3. Explain if-elif-else, for and while loops in Python with programs.
4. Explain functions in Python with default, keyword and variable-length arguments.
5. Write Python programs: prime number, factorial, Fibonacci series, largest of three numbers, simple interest.

UNIT 5 Python Data Structures, Files and Libraries

✍️ 2-mark definitions

- **List:** an ordered, mutable collection in square brackets: `[10, 20, 30]`.
- **Tuple:** an ordered, immutable collection in round brackets: `(1, 2, 3)`. Faster than a list and can be a dict key.
- **Dictionary:** an unordered (insertion-ordered since Python 3.7) collection of key-value pairs: `{"roll": 1, "name": "Asha"}`. Keys must be unique and immutable.
- **Set:** an unordered collection of unique items: `{1, 2, 3}`. Supports union, intersection and difference.
- **Slicing:** getting part of a sequence with `[start:stop:step]`; stop is excluded.
- **List comprehension:** a short way to build a list: `[x*x for x in range(5)]`.
- **CSV file:** comma-separated values, a plain-text table read with the `csv` module or Pandas.
- **NumPy:** a library for fast arrays and maths on whole arrays at once. **Matplotlib:** a library for plotting graphs.

💡 Core concepts

Comparison:

Feature	List	Tuple	Set	Dictionary
Brackets	[ ]	( )	{ }	{key: value}
Ordered	Yes	Yes	No	Yes (insertion)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	By position	By position	No	By key

Common methods:

- **List:** append, insert, extend, remove, pop, sort, reverse, index, count.
- **Dict:** keys, values, items, get, update, pop.
- **Set:** add, remove, union (|), intersection (&), difference (−).
- **String:** upper, lower, split, join, strip, replace, find.

File handling:

```
with open("marks.txt", "w") as f:      # 'with' closes the file automatically
    f.write("Asha 92\nRavi 85\n")

with open("marks.txt") as f:
    for line in f:
        name, m = line.split()
        print(name, int(m))
```

CSV:

```
import csv
with open("data.csv", newline="") as f:
    for row in csv.reader(f):
        print(row)               # each row is a list of strings
```

NumPy and Matplotlib:

```
import numpy as np
import matplotlib.pyplot as plt

a = np.array([1, 2, 3])
print(a * 2, a.mean())          # [2 4 6] 2.0

hours = np.arange(0, 24, 3)
temp = 25 + 5 * np.sin(hours / 24 * 2 * np.pi)
plt.plot(hours, temp, marker="o")
plt.xlabel("Hour"); plt.ylabel("Temperature (°C)"); plt.title("Daily temperature")
plt.show()
```

IoT sensor simulation (typical exam program):

```
import random
readings = []
for t in range(10):
    value = round(random.uniform(20, 35), 1)   # simulated temperature
    readings.append(value)
    if value > 30:
        print(f"t={t}s  {value} °C  ALERT: high temperature")
print("Average:", sum(readings) / len(readings))
```

Output questions

```
L = [10, 20, 30, 40, 50]
print(L[1:4], L[::-1], L[-2])
```

Output: **[20, 30, 40] [50, 40, 30, 20, 10] 40**.

```
s = {1, 2, 2, 3}
print(s, {1, 2, 3} & {2, 3, 4}, {1, 2, 3} | {3, 4})
print([x * x for x in range(1, 6) if x % 2])
```

Output: **{1, 2, 3} {2, 3} {1, 2, 3, 4}** and **[1, 9, 25]**.

```
d = {'a': 1, 'b': 2}
d['c'] = 3
print(d, list(d.keys()))
```

Output: **{'a': 1, 'b': 2, 'c': 3} ['a', 'b', 'c']**.

Key points to remember

- List mutable, tuple immutable, set unique, dict key-value.
- Slicing excludes stop; [::-1] reverses.
- Use `with open(...)` so files close automatically.
- NumPy works on whole arrays; Matplotlib plot, xlabel, ylabel, title, show.

⭐ Important questions

1. Compare list, tuple, set and dictionary with examples and methods.
2. Explain slicing and list comprehension; write a program to count word frequency using a dictionary.
3. Explain file handling in Python; write a program to read a CSV of marks and print the average.
4. What is NumPy? Write a program to create arrays and do basic operations. Plot a line graph using Matplotlib.
5. Write a program to simulate IoT sensor readings, raise an alert above a threshold and plot the values.

Last-hour checklist

☐ C program structure; compile-link steps; data type table; operator table and precedence; outputs 3 1 3.50 0, 20 5 67 C, 17.

☐ if, switch with break, loop table; call by value vs reference (swap); recursion fact(5) = 120, fib(10) = 55; prime, Armstrong 153, calculator.

☐ Arrays (max 9), matrix add and transpose; string functions; pointers (*q = 20 changes p); structure vs union; file modes; malloc vs calloc.

☐ Python features; C vs Python table; // and **; outputs 3 1 3.5 1024 −4; default arguments; prime, factorial, Fibonacci 0 to 34.

☐ List, tuple, set, dict table; slicing outputs; set operations; files with `with`; CSV; NumPy [2 4 6] 2.0; Matplotlib; IoT simulation.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Problem Solving and Programming in C	S. Sridhar, V. M. Hariharan	Pearson	WorldCat
2	Basic Computer Engineering	Sanjay Silakari, Rajesh K. Shukla	Wiley India	WorldCat
3	Python Programming	S. Sridhar, J. Indumathi, V. M. Hariharan	Pearson	WorldCat
4	Let Us C	Yashavant Kanetkar	BPB Publications	WorldCat
5	Python Crash Course	Eric Matthes	No Starch Press	WorldCat

Syllabus source: AKTU B.Tech 1st year CSE stream course structure and syllabus, 2026-27 (official PDF)

NEXT ENGINEER _sheerat_